

SHM Machine Pricing Case – Data Science Report

Overview of the data set

The **Bit SHM data set** contains 412 698 auction records for second-hand machinery from 1989 to April 2012. Each row describes an auction and includes a unique `Sales ID`, a target variable `Sales Price` (continuous), the machine `Model ID`, manufacture year, hours on the meter, a categorical usage band and many other descriptors such as product class, enclosure type, transmission, hydraulics and so on. There are 54 columns in total. The training set is large (~400 k rows), and, like the Kaggle **Blue Book for Bulldozers** competition on which it is based, the data are messy and heterogeneous.

Missing-value patterns

An initial inspection shows that some columns have extremely high missing rates. For example, variables such as `Clean Room`, `Engine Horsepower`, `Machine Width`, `Extra features`, `Push Block` and `Tip Control` are missing in ~94 % of the records. Many other columns (e.g. `Grouser Tracks`, `Hydraulics Flow`, `Couple System`, `Model Series`) are missing in 80–90 % of the cases. Conversely, key identifiers (`Sales ID`, `Machine ID`, `Model ID`) and the target variable are complete. A few technical variables contain special placeholder values: the book on the bulldozer competition notes that the `YearMade` column sometimes contains the value **1000**, which indicates “unknown” and should be replaced with missing ¹. Likewise, machine hours equal to zero often mean the hours are unknown rather than that the machine is brand new ². These anomalies must be treated explicitly to avoid misleading the model.

Time-dependence

The auctions span more than two decades. Because price levels drift over time due to inflation and market cycles, the data exhibit temporal non-stationarity. The fast.ai notes on the dataset warn that standard random splits or out-of-bag scores can overestimate performance and recommend sorting by date and taking the last ~20 % of the data as a hold-out set ³. This ensures the model is evaluated on “future” data and better reflects its ability to generalise to unseen auctions.

Target distribution

`Sales Price` ranges from about 4 750 to 142 000 USD, with a mean of ~31 200 and a heavy right tail (skewness ≈ 1.5). The skewness suggests that applying a logarithmic transform (or using **RMSLE** as the loss metric) can help stabilise variance and penalise large relative errors more evenly.

Features that stand out

1. **Manufacture year** (`Year Made`) – a strong driver of value. Many records use **1000** as a sentinel indicating an unknown year. The fast.ai analysis recommends replacing `Year Made` < 1950 with

missing and imputing later ¹. Some records even have sale dates earlier than the manufacture date; these should be corrected by setting the manufacture year to the sale year ⁴.

2. **Sale date** – includes both a date string and a timestamp. Converting this to numeric features (sale year, month, day-of-year) enables the model to capture seasonality and macro-economic effects. Since prices drift over time ³, one should also compute the machine's **age at sale** (sale year minus manufacture year).
3. **MachineHoursCurrentMeter** – the hours on the meter correlate with wear and therefore price. However, ~64 % of the values are missing and zeros often denote unknown hours ². Consider creating an indicator variable for “hours missing” and imputing the numeric part with the median per model or product class.
4. **High-cardinality categories** – columns such as `Model Description`, `Secondary Description`, `Product Class Description` and `Product Group` have hundreds to thousands of unique values. Treating these as categorical variables with one-hot encoding would explode dimensionality. Gradient-boosting methods that handle categorical variables internally (e.g. CatBoost) or techniques like **target encoding**/ **frequency encoding** should be used.
5. **Highly missing options** – variables describing rare options (e.g. `Tip Control`, `Push Block`, `Extra features`) are missing in >90 % of cases. Including them as binary features may add noise. Consider dropping them or replacing missing with “Unknown” and letting the model learn any small effect, but be wary of over-fitting.

Proposed preprocessing steps

1. **Parsing dates** – convert the `Sales date` string to a proper datetime and derive `sale_year`, `sale_month` and `sale_dayofyear`. Compute the **machine age** as `sale_year - Year Made`. Then drop the original date column.
2. **Handling sentinel values** – treat `Year Made` < 1900 (the dataset uses 1000) as missing ¹. Replace machine hours equal to zero with missing ². For impossible cases where the sale precedes the manufacture year, set `Year Made` to the sale year ⁴.
3. **Missing-value imputation** – follow the fast.ai recommendation ⁵: for each numeric column with missing values, create a companion boolean column (e.g. `YearMade_na`) marking missingness and replace missing values with the median. For categorical columns, replace missing values with a special category `Unknown` so the model can learn from the pattern of missingness.
4. **Categorical encoding** – use models that natively handle categorical variables such as **CatBoost** or **LightGBM** with `categorical_feature` arguments. Alternatively, encode high-cardinality features via **target encoding** (mean price per category) or **frequency encoding** to reduce dimensionality.
5. **Outlier management and scaling** – because the target is skewed, log-transform the sale price when training or use **RMSLE** as the objective. Clip negative predictions at 1 when computing RMSLE

to avoid domain errors. Standardisation of numeric features is not critical for tree-based models but may help for linear models.

Model choice and justification

Tree-based ensembles are well suited to this problem. They capture non-linear interactions, handle missing values and categorical variables, and scale to hundreds of thousands of observations. Three candidates are:

1. **CatBoost Regressor** – natively supports categorical variables without one-hot encoding, handles missing values gracefully, and often performs well with little tuning. A baseline model with ~150 trees trained on a 12 % sample achieved an RMSE of ~8 600 USD and RMSLE ≈ 0.267 on a hold-out set.
2. **LightGBM Regressor** – efficient gradient boosting that can treat columns as categorical using integer codes. LightGBM is faster than CatBoost on large datasets and allows tuning of leaf growth strategies. However, categories must be integer-encoded and missing values handled beforehand.
3. **Random Forest / Extra Trees** – provide robust baselines and are easy to interpret. The Dataiku team's solution to the original contest used random forests on different product groups and tuned hyper-parameters ⁶. Random forests handle many variables but can struggle with high cardinality and long training times.

Linear models with regularisation (ridge, lasso) were less effective in the Kaggle competition because the relationships are non-linear and interactions between options are complex. Neural networks could work but require more tuning and encoding of categoricals.

Model evaluation plan

1. **Temporal split** – sort the data by `Sales date` and hold out the most recent 20 % of auctions for validation ³. Do not randomly shuffle because price dynamics change over time. If the dataset is later used in production, update the model periodically as new sales arrive.
2. **Evaluation metrics** – use **Root Mean Squared Logarithmic Error (RMSLE)** as the primary metric (the Kaggle competition used this), which treats relative errors equally and dampens the influence of outliers. Also report RMSE on the original scale for interpretability. Because `Sales Price` is strictly positive, clip predictions at 1 before computing RMSLE.
3. **Cross-validation** – perform **time-series cross-validation** with several folds, each using earlier years to predict later years. This assesses stability of the model across time and reduces the risk of over-fitting to a single period.
4. **Hyper-parameter tuning** – perform grid or Bayesian optimisation on key parameters (number of trees, learning rate, maximum depth, and regularisation) using the validation set. Monitor feature importance to detect over-fitting and prune uninformative variables.

5. **Error analysis** – inspect residuals to identify systematic biases (e.g. under-valuing certain models or high-hour machines). Investigate whether some product groups or states of usage consistently produce large errors and consider training specialised models per group ⁷.

Rough time allocation (4 hours total)

Activity	Description	Estimated time
Data loading & preliminary exploration	Inspect the CSV, check column types, summarise missingness and distributions; verify assumptions such as sentinel values and duplicate entries.	0.5 h
Feature engineering & cleaning	Parse dates, compute age, replace sentinel values, impute missing numeric values with medians and create missing-indicator variables, set unknown categories. Drop extremely sparse variables or consolidate rare categories.	1.0 h
Model selection & initial training	Set up CatBoost/LightGBM baselines, select categorical features, train on a sampled subset to iterate quickly, monitor RMSE and RMSLE. Experiment with log-transform of the target.	1.0 h
Hyper-parameter tuning & validation	Perform time-series cross-validation, tune learning rate, depth and regularisation. Assess performance across folds; adjust feature engineering if necessary (e.g. add market trend or per-model means).	0.8 h
Evaluation & error analysis	Compare RMSE and RMSLE on hold-out; inspect residuals; analyse under- and over-estimation. Consider training per product group to capture heterogeneity.	0.4 h
Reporting	Summarise methodology, justify design choices, and document results in a concise report. Package the cleaned code/notebook and prepare discussion points for stakeholders.	0.3 h

Conclusion and recommendations

The SHM auction data present a complex regression problem with heterogeneous features, heavy missingness and temporal drift. Careful preprocessing is crucial: treat sentinel values (e.g. `Year Made` = 1000) as missing ¹, derive temporal features and machine age, and handle the high cardinality of categorical variables. Tree-based models such as CatBoost or LightGBM are appropriate because they capture interactions and require minimal encoding. Evaluation should respect the time-series nature of the data by holding out recent sales ³, and RMSLE should be the primary metric. Finally, iterate creatively – the Dataiku team improved performance by adding day-of-week features, computing the number of auctions in the current month and training separate models per product group ⁸. This case is an opportunity to experiment with different feature representations and model architectures while always grounding decisions in domain knowledge and the data's inherent structure.

1 2 3 4 5 Exploring and Cleaning the Bulldozer Dataset

<https://mlbook.explained.ai/bulldozer-intro.html>

6 7 8 Kaggle Contest: Blue Book For Bulldozers

<https://blog.dataiku.com/2013/04/26/kaggle-contest-blue-book-for-bulldozers>